Foldable & Traverse

son: 6 · FP Foundations · Category: Functional · Level: Intermediate · Build with Wa

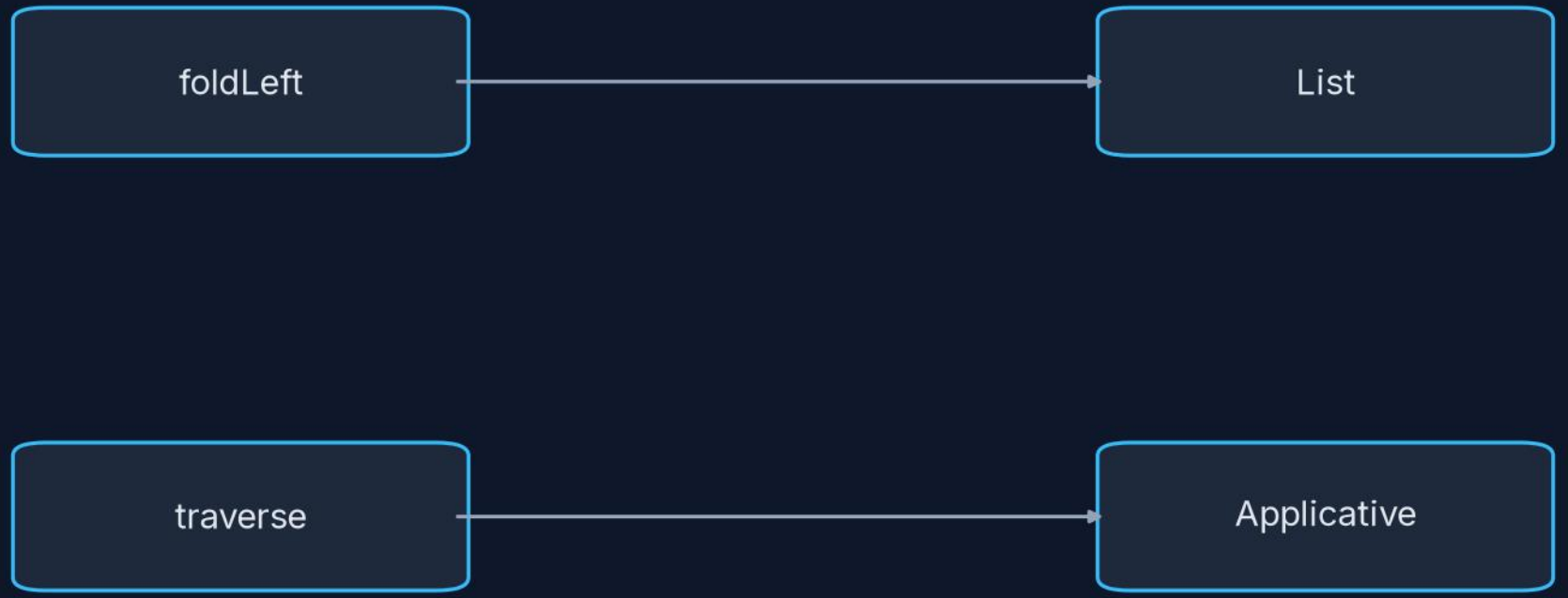
Today's Hook

- Today: Foldable & Traverse as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Foldable & Traverse structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Foldable & Traverse
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/functional/foldable/Foldable.java
package com.buildwithwaqar.patterns.functional.foldable;

import com.buildwithwaqar.patterns.functional.monoid.Monoid;
import com.buildwithwaqar.patterns.functional.typeclass.Applicative;
import com.buildwithwaqar.patterns.functional.typeclass.Kind;

import java.util.ArrayList;
import java.util.List;
import java.util.function.BiFunction;
import java.util.function.Function;

/** Foldable + Traverse for List — educational encodings. */
public final class Foldable {
    private Foldable() {}

    public static <A, B> B foldLeft(List<A> xs, B z, BiFunction<B, A, B> f) {
        B acc = z;
        for (A x : xs) acc = f.apply(acc, x);
        return acc;
    }
```

Implementation (2)

```
    public static <A> A fold(List<A> xs, Monoid<A> m) {
        return foldLeft(xs, m.empty(), m::combine);
    }

    public static <F, A, B> Kind<F, List<B>> traverse(Applicative<F> A, List<A> xs, Function<A, Kind<F, B>> f) {
        Kind<F, List<B>> acc = A.pure(new ArrayList<>());
        for (A x : xs) {
            Kind<F, B> fb = f.apply(x);
            acc = A.ap(A.map(acc, list -> (b -> { list.add(b); return list; })), fb);
        }
        return acc;
    }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.functional.foldable;
import com.buildwithwaqar.patterns.functional.monoid.Monoids;
import java.util.List;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Foldable walkthrough ===");
    System.out.println("foldLeft=" + Foldable.foldLeft(List.of(1,2,3), 0, Integer::sum));
    System.out.println("foldMonoid=" + Foldable.fold(List.of(1,2,3), Monoids.INT_SUM));
  }
}
```

Walkthrough Output

```
=== Foldable walkthrough ===
foldLeft=6
foldMonoid=6
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Foldable & Traverse
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Foldable & Traverse → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.